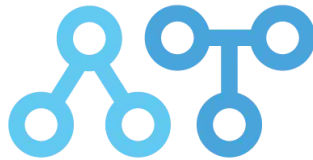




MEMORIA TÉCNICA

Proyecto Final · 2º DAW



alan TURING

CENTRO PÚBLICO INTEGRADO DE FORMACIÓN PROFESIONAL

Cristina Fernández Fernández

Desarrollo de Aplicaciones Web

HTML5	JavaScript	React	Node.js
MongoDB	Express	Socket.io	JWT + bcrypt

Contents

1. Introducción.....	3
Secciones principales.....	3
2. Objetivos.....	4
Funcionalidades implementadas.....	4
Líneas de mejora futuras.....	4
3. Tecnologías y Justificación.....	5
4. Diseño de la Aplicación.....	6
4.1 Figma y wireframes.....	6
4.2 Guía de estilos.....	6
Tipografía.....	6
4.3 Decisiones UX.....	6
4.4 Estructura de carpetas.....	7
5. Arquitectura de la Aplicación.....	8
5.1 Base de datos — MongoDB.....	8
5.2 Configuración del servidor — server.js.....	9
5.3 Endpoints de la API REST.....	9
5.4 Función mostrarDestinos — Frontend.....	10
5.5 Modal de destinos.....	12
5.6 Autenticación JWT.....	13
Registro e inicio de sesión.....	13
5.7 Módulo de Experiencias — React.....	14
5.8 Chat en tiempo real — Socket.io.....	17
5.9 Conversor de divisas — ExchangeRate API.....	18
5.10 Página 404 personalizada.....	18
5.11 Seguridad y Clean Code.....	18
6. Manual de Despliegue.....	19
Requisitos.....	19
Pasos.....	19

1. Introducción

ViajaMás es una plataforma web full-stack pensada para conectar viajeros. Permite explorar destinos con filtros, convertir divisas, compartir experiencias con fotos, chatear en tiempo real y gestionar sesiones de forma segura.

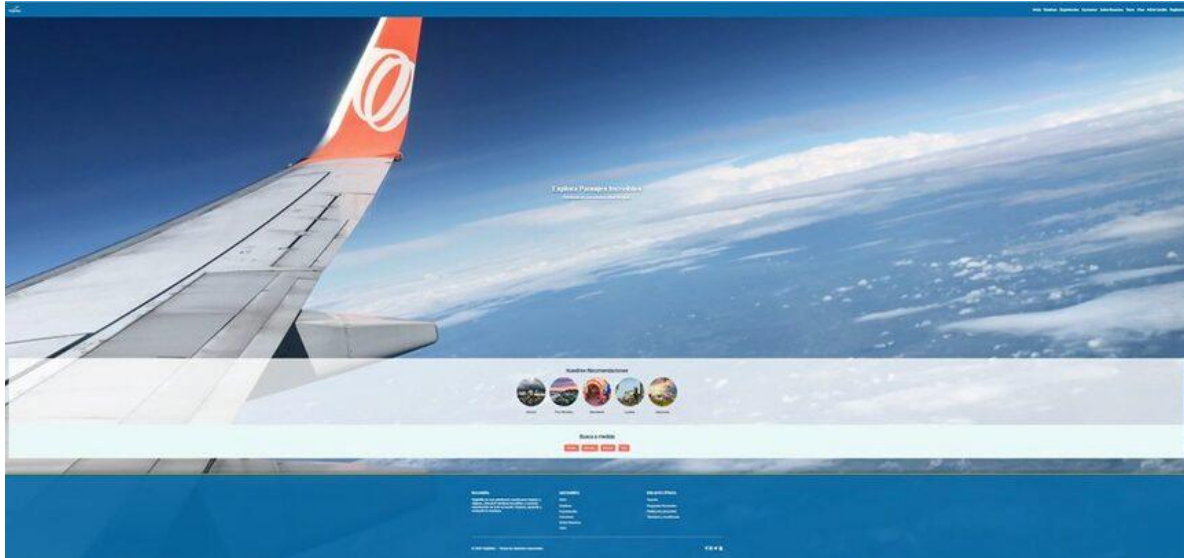


Figura 1 — Página principal con banner de vuelo y recomendaciones

Secciones principales

- **Inicio:** Banner con vídeo, carrusel de destinos recomendados y filtros de búsqueda.
- **Destinos:** Grid de tarjetas con imagen, moneda, idioma y descripción. Modal con navegación por teclado.
- **Experiencias:** Muro de comentarios con valoración por estrellas y subida de fotos a Cloudinary. Requiere autenticación.
- **Chat:** Comunicación en tiempo real entre usuarios autenticados mediante Socket.io.
- **Conversor de divisas:** Integración con ExchangeRate API. Más de 160 monedas, conversión instantánea.
- **Autenticación:** Registro con bcrypt, login con JWT de 24h, cierre de sesión y redirección automática.



Figura 2 — Navbar con todas las secciones de ViajaMás

2. Objetivos

El objetivo es construir una aplicación real de nivel profesional que integre frontend dinámico, backend seguro, base de datos en la nube, almacenamiento multimedia y comunicación en tiempo real en un único proyecto cohesionado.

Funcionalidades implementadas

- Arquitectura MVC completa en el backend (Node.js + Express)
- Autenticación JWT con protección de rutas y extracción de identidad desde el token
- CRUD de comentarios con subida de imágenes a Cloudinary
- Chat en tiempo real con Socket.io (mensajes propios a la derecha, estilo WhatsApp)
- Conversor de divisas con API externa (ExchangeRate API)
- Filtros por nombre, moneda e idioma con query parameters
- Página 404 personalizada con animación CSS temática de vuelo
- Variables de entorno (.env) y exclusión de credenciales con .gitignore

Líneas de mejora futuras

- Crear un E-Commerce de venta de productos para viajar
- Despliegue en producción

3. Tecnologías y Justificación

He optado por el stack MERN porque unifica JavaScript en todas las capas, facilitando el intercambio nativo de datos JSON sin adaptadores entre capas.

Tecnología	Justificación técnica
HTML5 + CSS3	Estructura semántica y estilos. Paleta corporativa definida en variables CSS.
JavaScript ES6+	Lógica de filtros, modales, conversor de divisas y autenticación en el cliente.
React (Vite)	Módulo de Experiencias con hooks useState/useEffect. Virtual DOM para renders optimizados.
Node.js + Express	Servidor backend con arquitectura MVC. Gestión de rutas, middlewares y Socket.io.
MongoDB	Base de datos NoSQL. Flexible para documentos JSON. 4 colecciones: destinos, usuarios, comentarios.
Socket.io	WebSockets para el chat en tiempo real. Comunicación bidireccional sin polling HTTP.
JWT + bcrypt	Autenticación stateless con tokens firmados. Contraseñas hasheadas con salt automático.
Cloudinary	Almacenamiento de imágenes en la nube. multer + multer-storage-cloudinary. URLs permanentes y optimizadas.

💡 Un único lenguaje en frontend y backend reduce la curva de aprendizaje, homogeneiza los tipos de datos y permite reutilizar validaciones sin duplicar código.

4. Diseño de la Aplicación

4.1 Figma y wireframes

El diseño se planificó primero en Figma antes de empezar a codificar, definiendo la estructura de todas las páginas y la identidad visual.

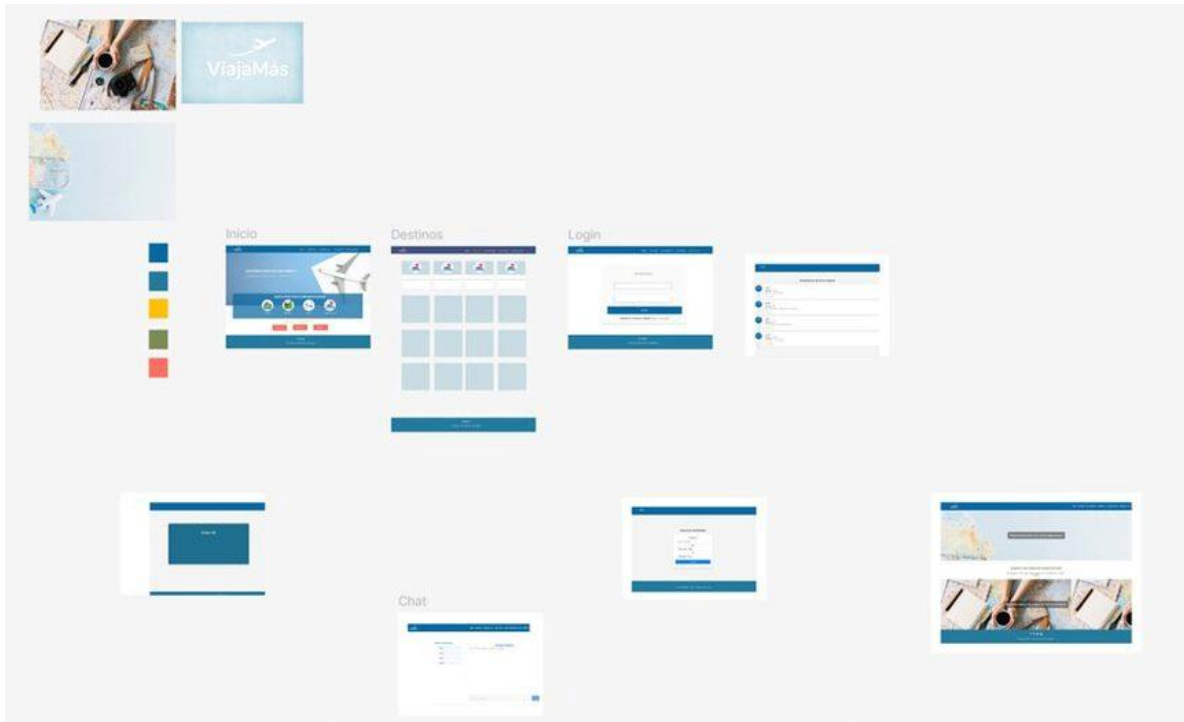


Figura 3 — Wireframes en Figma: inicio, destinos, login, chat y experiencias

4.2 Guía de estilos

Azul principal #0a6aa6	Azul oscuro #084f80	Amarillo CTA #FFC107	Verde #7D8C57	Fondo claro #e8f0ff
------------------------	---------------------	----------------------	---------------	---------------------

Tipografía

- **Títulos:** Poppins / Montserrat Bold 700. Espaciado generoso para transmitir amplitud.

4.3 Decisiones UX

- Parallax scroll en Experiencias para usuarios no autenticados, generando engagement visual.
- Efecto blur (backdrop-filter) en el chat para usuarios no logueados: mantiene el contexto pero bloquea la acción.
- Modal de destinos con navegación por teclado (←/→) para experiencia accesible sin ratón.

- Avatares generados con ui-avatars.com usando las iniciales y color corporativo, sin almacenamiento extra.
- Botón 'Publicar' con estado 'Publicando...' y deshabilitado durante el envío para prevenir doble submit.

4.4 Estructura de carpetas

```
ViajaMás/  
├── backend/  
│   ├── controllers/    ← Lógica de negocio  
│   ├── models/         ← db.js (conexión MongoDB singleton)  
│   ├── middleware  
│   ├── routes/         ← Endpoints REST  
│   ├── chatLogic.js    ← Socket.io  
│   └── server.js       ← Punto de entrada  
├── frontend-react/src/components/Experiencias.jsx  
├── pages/              ← HTML estático  
├── js/                 ← Scripts frontend  
├── 404.html            ← Error con animación CSS  
└── .env               ← Credenciales (excluido de Git)
```

5. Arquitectura de la Aplicación

5.1 Base de datos — MongoDB

La base de datos viajaMas contiene cuatro colecciones. Las imágenes de destinos se almacenan en Cloudinary y solo se guarda la URL en MongoDB.

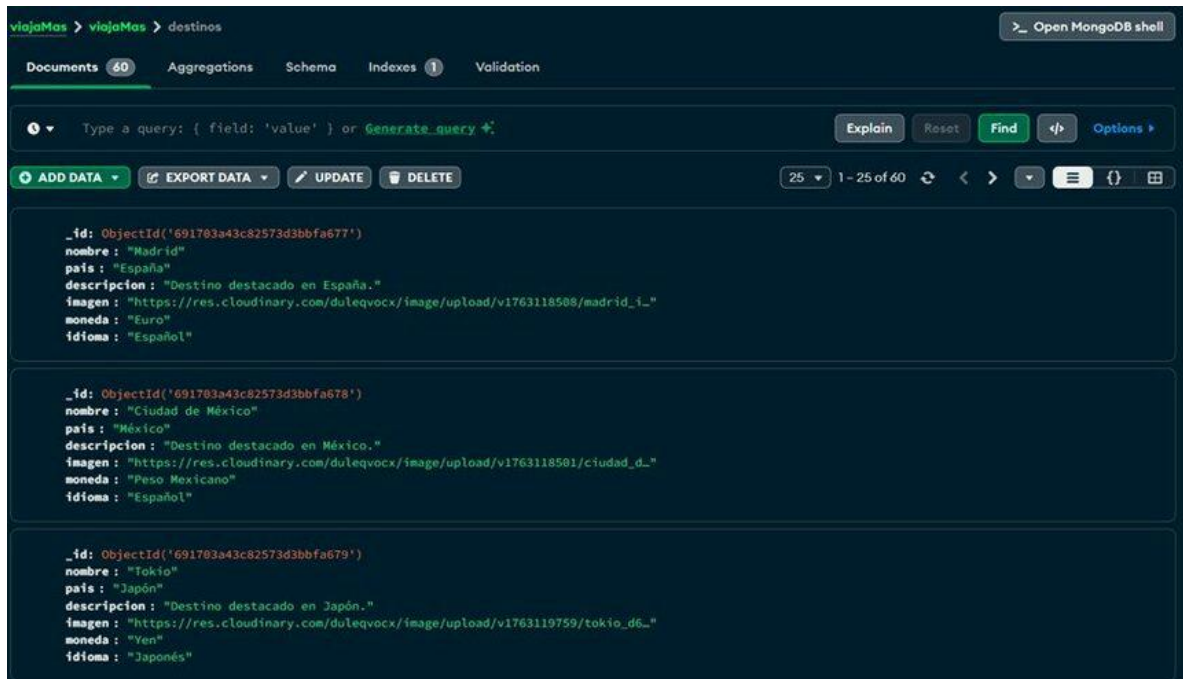


Figura 4 — Colección destinos en MongoDB Compass con 60 documentos



Figura 5 — Librería de imágenes de destinos en Cloudinary (103 assets)

5.2 Configuración del servidor — server.js

El servidor orquesta todos los módulos.

```
1  const express = require("express");
2  const { MongoClient } = require("mongodb");
3  const cors = require("cors");
4
5  const app = express();
6  const PORT = 3000;
7  const MONGO_URI = "mongodb://localhost:27017";
8  const DB_NAME = "viajaMas";
9
10 app.use(cors());
11 app.use(express.json());
12
13 async function conectarDB(nombreColeccion) {
14   const client = new MongoClient(MONGO_URI);
15   await client.connect();
16   console.log('✅ Conectado a la colección: ${nombreColeccion}');
17   return client.db(DB_NAME).collection(nombreColeccion);
18 }
19
20 // Obtener paises
21 app.get("/paises", async (req, res) => {
22   try {
23     const collection = await conectarDB("pais");
24     let filtro = {};
25
26     if (req.query.nombre) filtro.nombre = { $regex: req.query.nombre, $options: "i" };
27     if (req.query.moneda) filtro.moneda = { $regex: req.query.moneda, $options: "i" };
28     if (req.query.idioma) filtro.idioma = { $regex: req.query.idioma, $options: "i" };
29
30     const paises = await collection.find(filtro).toArray();
31     res.json(paises);
32   } catch (error) {
33     console.error("❌ Error en MongoDB:", error);
34     res.status(500).json({ error: "Error al obtener los datos." });
35   }
36 });
```

Figura 6 — server.js inicial con Express, MongoDB y filtros de búsqueda

⚠ La conexión MongoDB se abría en cada petición en la versión inicial. Se refactorizó a un patrón singleton (client global) para reutilizar la conexión y evitar saturación bajo carga.

5.3 Endpoints de la API REST

Todos los endpoints se probaron con Postman antes de integrar el frontend.

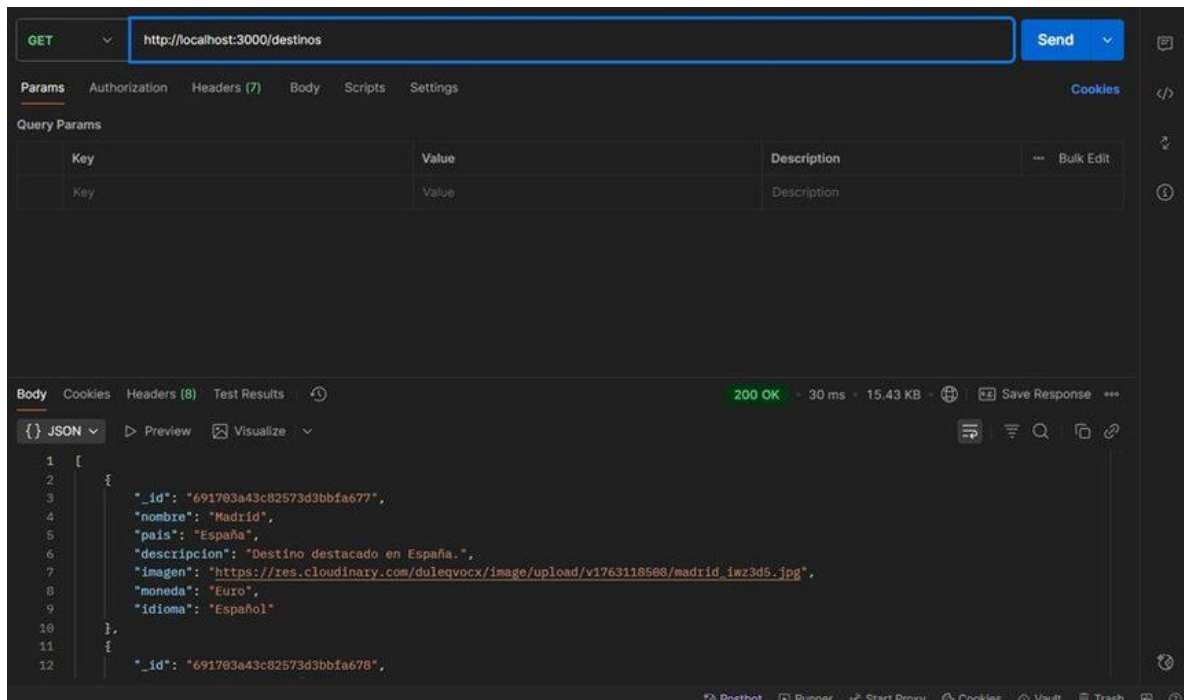


Figura 7 — Postman: GET /destinos devuelve 200 OK con los documentos de MongoDB

Método	Endpoint	Auth	Descripción
GET	/destinos	No requerida	Lista destinos con filtros opcionales por nombre, moneda e idioma
POST	/usuarios/registro	No requerida	Registra usuario, hashea contraseña con bcrypt
POST	/usuarios/login	No requerida	Autentica y devuelve JWT con 24h de validez
GET	/comentarios	No requerida	Obtiene todos los comentarios del muro
POST	/comentarios	JWT requerido	Crea comentario. Nombre extraído del token, no del formulario

5.4 Función mostrarDestinos — Frontend

La función mostrarDestinos(listaDestinos) genera dinámicamente las tarjetas de destinos con innerHTML, gestionando el caso de que idioma sea un array.

```
function mostrarDestinos(listaDestinos) {
  contenedor.innerHTML = "";

  listaDestinos.forEach((destino, index) => {

    const destinoCard = document.createElement("div");
    destinoCard.classList.add("pais-card");

    destinoCard.innerHTML = `
      
      <h3>${destino.nombre}</h3>
      <div class="detalles">
        <p><strong>Moneda:</strong> ${destino.moneda}</p>
        <p><strong>Idioma:</strong> ${Array.isArray(destino.idioma) ? destino.idioma.join(", ") : destino.idioma}</p>
        <p><strong>Descripción:</strong> ${destino.descripcion}</p>
      </div>
    `;

    destinoCard.addEventListener("click", () => {
      indexActual = index;
      mostrarModal(destinos[indexActual]);
    });
  });
}
```

Figura 8 — Función `mostrarDestinos` con `Array.isArray` para el campo `idioma`

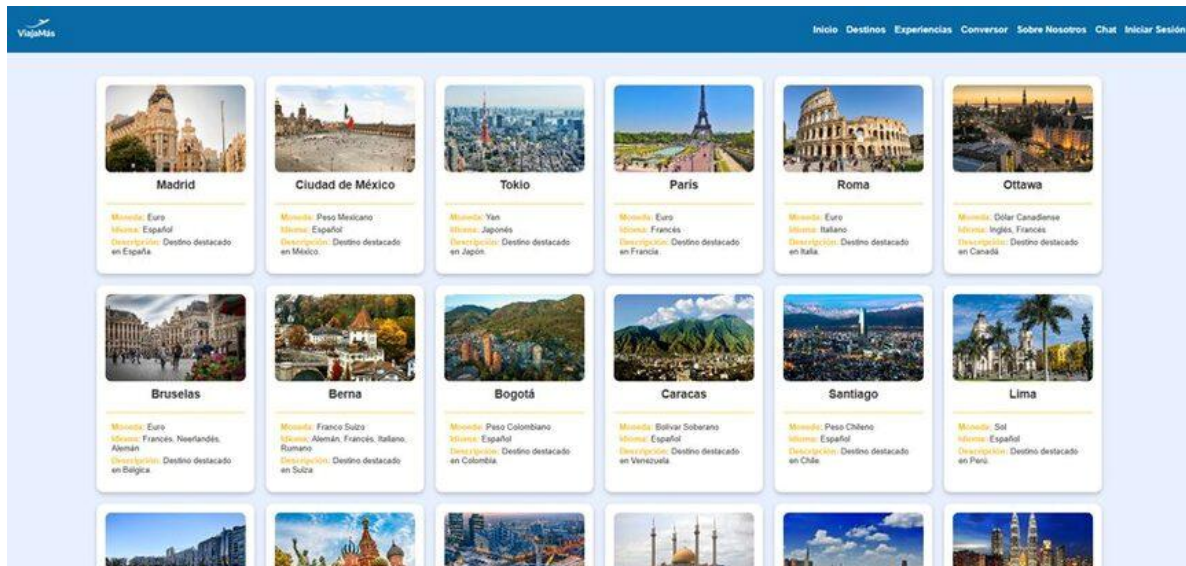


Figura 9 — Grid de destinos con tarjetas, imagen, moneda, idioma y descripción

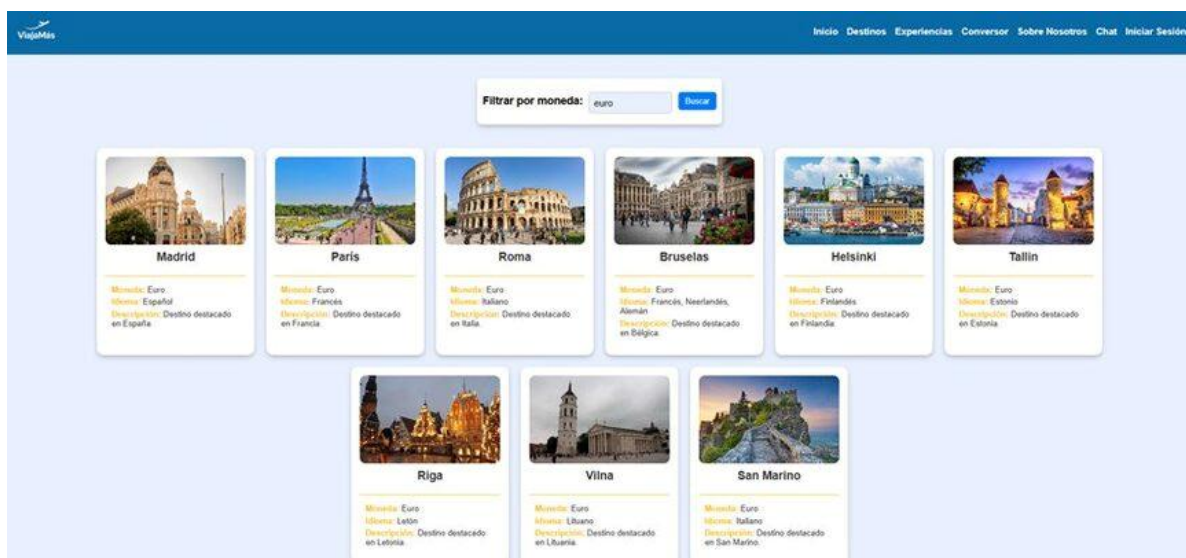


Figura 10 — Filtro por moneda: query parameter `?moneda=euro` en tiempo real

5.5 Modal de destinos

Al hacer clic en una tarjeta se abre un modal con la imagen ampliada. Implementa navegación por teclado con flechas y cierre al clic fuera.



Figura 11 — Tarjeta de Berna con datos en amarillo corporativo

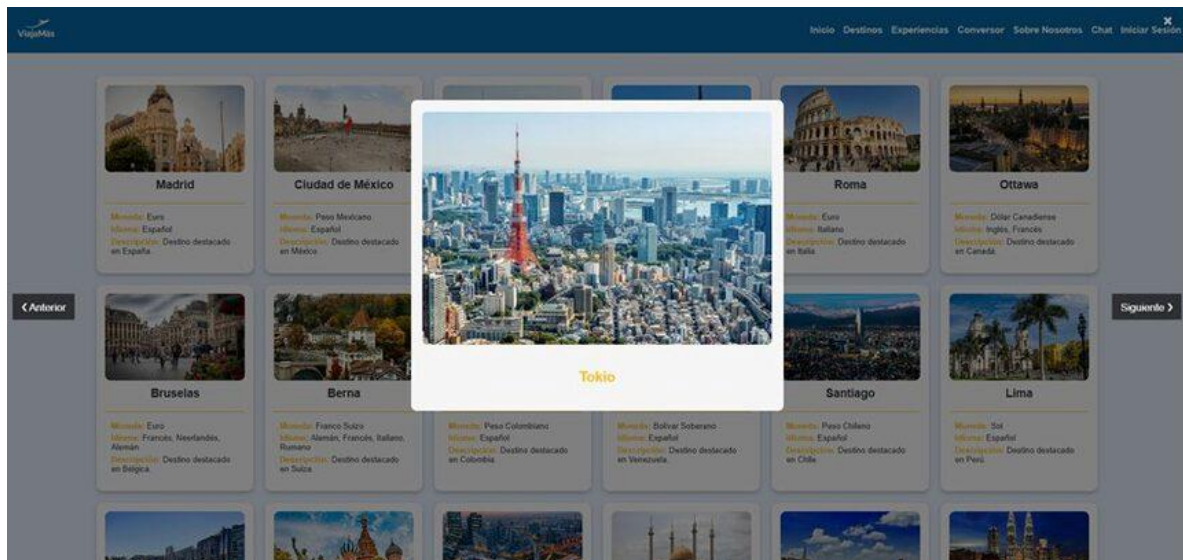


Figura 12 — Modal de Tokio con botones anterior/siguiente y overlay

5.6 Autenticación JWT

Registro e inicio de sesión

Figura 13 — Formulario de registro con validación de campos

Figura 14 — Formulario de inicio de sesión

1. El cliente envía email + contraseña al endpoint POST /usuarios/login
2. El servidor verifica el hash con `bcrypt.compare()`
3. Si coincide, genera un JWT firmado con `JWT_SECRET` (24h de expiración)
4. El cliente guarda el token en `localStorage` y lo envía en `Authorization: Bearer <token>`
5. El servidor verifica y decodifica el token sin consultar la base de datos

 Seguridad clave: el nombre del autor en los comentarios se extrae del JWT verificado en el servidor, no del formulario del cliente. Esto elimina la posibilidad de suplantación de identidad.

5.7 Módulo de Experiencias — React

Interfaz implementada como componente funcional (`Experiencias.jsx`). Requiere sesión activa — si el usuario no está autenticado se muestra un parallax con llamada a la acción.



Figura 15 — Parallax de Experiencias para usuarios no autenticados



Figura 16 — Footer con parallax y llamada al registro

- **useState** — lista de comentarios, datos del formulario, archivo adjunto y estado enviando.
- **useEffect(fn, [])** — ejecuta cargarComentarios() una sola vez al montar (array vacío = sin redisparos).
- **FormData** — permite enviar archivos binarios (imagen) junto con campos de texto en una sola petición.

```
// Protección anti doble-submit
const handleSubmit = (e) => {
  e.preventDefault();
  if (enviando) return; // bloquea si ya hay envío en curso
  setEnviando(true);
  // ... fetch al backend ...
  .finally(() => setEnviando(false)); // siempre desbloquea
};
```

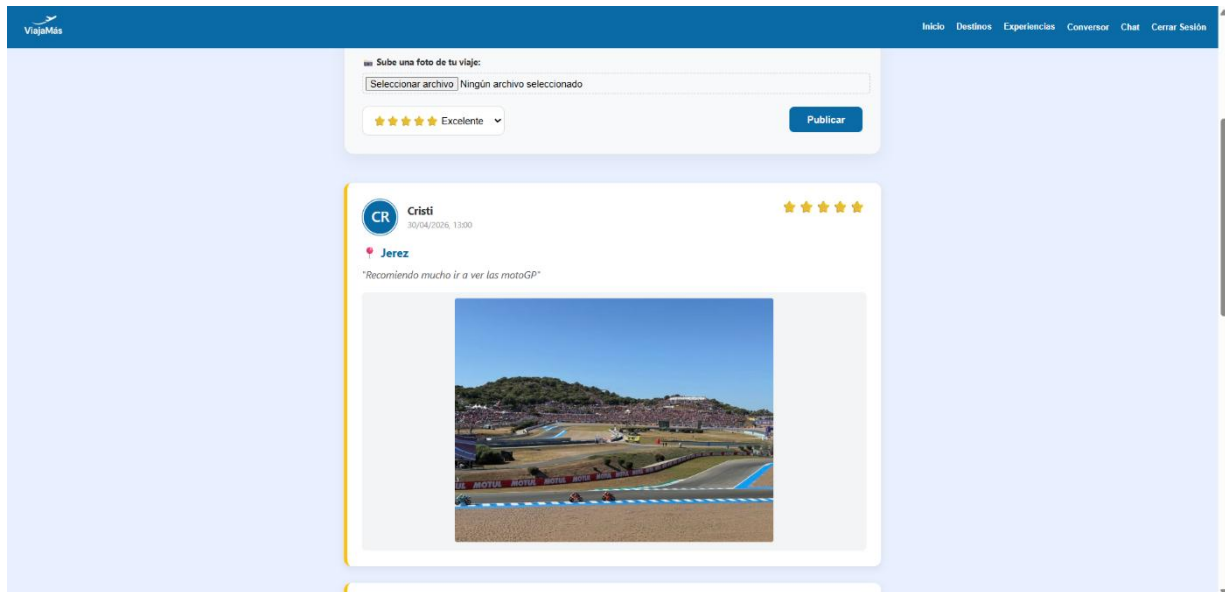


Figura 17 — Comentario publicado en el muro de experiencias

5.8 Chat en tiempo real — Socket.io

Comunicación bidireccional cliente-servidor sin polling HTTP repetido.

```
// chatLogic.js
module.exports = (io) => {
  io.on('connection', (socket) => {
    socket.on('mensaje_chat', (data) => {
      io.emit('mensaje_chat', data); // broadcast a todos
    });
  });
};
```

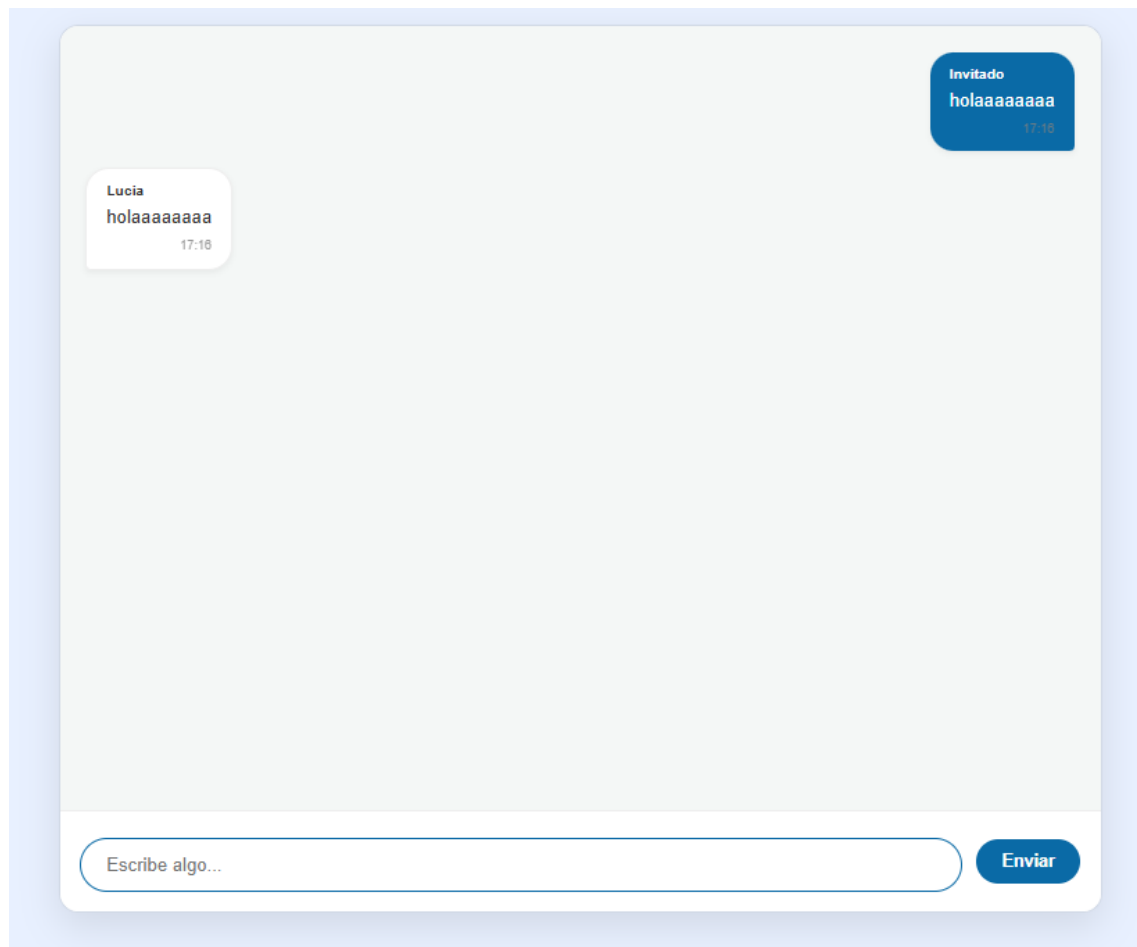


Figura 18 — Chat en tiempo real

5.9 Conversor de divisas — ExchangeRate API

- **GET /codes** — carga +160 monedas al iniciar la página, rellena los selectores dinámicamente.
- **GET /pair/{origen}/{destino}/{cantidad}** — ejecuta la conversión y muestra el resultado al instante.

5.10 Página 404 personalizada

Middleware Express al final de server.js que intercepta rutas no resueltas. Distingue entre rutas de API (responde JSON) y rutas de página (responde el HTML animado con temática de vuelo).

```
app.use((req, res) => {
  const esAPI = ['/usuarios', '/destinos', '/comentarios']
    .some(r => req.originalUrl.startsWith(r));
  if (esAPI) return res.status(404).json({ error: 'No encontrado' });
  res.status(404).sendFile(path.join(__dirname, '../404.html'));
});
```

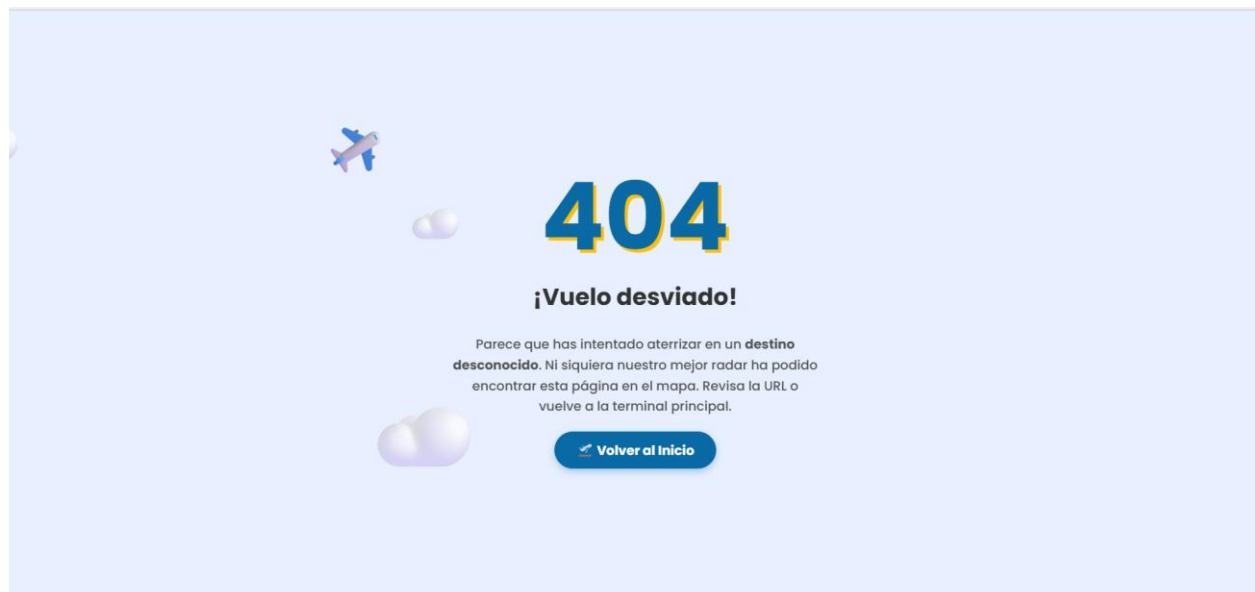


Figura 19 — Error 404 personalizado

5.11 Seguridad y Clean Code

- .env almacena MONGO_URI, JWT_SECRET y credenciales Cloudinary. Excluido de Git con .gitignore.
- CORS explícito: solo localhost:5500 y localhost:5173 en desarrollo.
- Validaciones previas a la BD: campos vacíos → HTTP 400, contraseñas < 6 caracteres rechazadas.

6. Manual de Despliegue

Requisitos

- Node.js v18 o superior
- MongoDB local en el puerto 27017 (o string de Atlas en el .env)
- Cuenta en Cloudinary con credenciales

Pasos

1. Clonar el repositorio:

```
git clone https://github.com/CristinaFdezFdez/Proyecto-final
```

2. Instalar dependencias:

```
cd Proyecto-final/backend  
npm install
```

3. Crear el archivo backend/.env:

```
MONGO_URI=mongodb://localhost:27017  
DB_NAME=viajaMas  
JWT_SECRET=tu_clave_secreta  
CLOUDINARY_CLOUD_NAME=tu_cloud_name  
CLOUDINARY_API_KEY=tu_api_key  
CLOUDINARY_API_SECRET=tu_api_secret
```

4. Arrancar el servidor:

```
node server.js
```

5. Abrir en el navegador:

```
http://localhost:3000
```

⚡ Importante: acceder siempre desde `http://localhost:3000`, no con Live Server. Solo así funcionan correctamente el middleware 404 y la autenticación de rutas.